

Lesson 20 Structured Note: Reusing a Linked-List Stack for Postfix Evaluation

0. Current Position in the Course

You are now in **Lesson 20** of the stack curriculum.

This is the final lesson in:

Phase 5: Postfix Evaluation and Integration

Before Lesson 20, you learned many important ideas:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack operations.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 7	A complete C linked-list stack uses <code>struct Node</code> , a global <code>top</code> pointer, <code>push</code> , <code>pop</code> , <code>display</code> , and <code>main</code> .
Lesson 8	A C++ class can be used to implement a linked-list stack more safely.
Lesson 9	Parenthesis matching was introduced as a stack application.
Lesson 10	Parenthesis matching was implemented using an array character stack.
Lesson 11	Parenthesis matching was implemented using a linked-list character stack.
Lesson 12	Bracket matching was extended to <code>()</code> , <code>[]</code> , and <code>{}</code> .
Lesson 13	You learned infix, prefix, postfix, precedence, and simple expression conversion.

Lesson	Main idea
Lesson 14	You learned associativity and unary operators.
Lesson 15	You learned the logic for converting infix to postfix using a stack.
Lesson 16	You wrote an infix-to-postfix program using an array character stack.
Lesson 17	You wrote an infix-to-postfix program using a linked-list character stack.
Lesson 18	You learned the theory and procedure of postfix evaluation.
Lesson 19	You wrote a postfix evaluation program using an integer stack.

Lesson 20 now connects two things you already know:

1. **Linked-list stack implementation**
2. **Postfix evaluation**

In simple words:

Lesson 20 teaches how to reuse a linked-list stack to evaluate a postfix expression.

1. Lesson 20 Goal

By the end of this lesson, you should understand:

- What it means to reuse a linked-list stack.
- Why postfix evaluation needs a stack.
- Why the stack must store integers during postfix evaluation.
- Why the node changes from `char data` to `int data`.
- How `push(int x)` works in the linked-list stack.
- How `pop()` works in the linked-list stack.
- Why `top == NULL` means the stack is empty.
- How to identify operands and operators.
- How `isOperand()` works.
- Why digit characters must be converted into integers.
- How `postfix[i] - '0'` works.
- Why pop order matters.
- Why we use `x2`, `x1`, and `r`.
- How the `Eval()` function works.

- How the `switch` statement performs arithmetic.
 - How to dry-run `234*+82/-`.
 - Why the final answer is left in the stack.
 - The time and space complexity of the solution.
 - Common beginner mistakes.
-

2. Big Idea of Lesson 20

The big idea is:

A stack can be reused for different problems if the stack behavior stays the same.

The stack behavior is still:

- `push` inserts at the top.
- `pop` removes from the top.
- The newest value is removed first.
- The stack follows **LIFO**.

But the stored data type can change.

Earlier, a stack may have stored characters.

For parenthesis matching, it stored characters like:

```
'('
')'
'['
']'
```

For infix-to-postfix conversion, it stored operators like:

```
'+'
'-'
'*'
 '/'
```

But for postfix evaluation, we are doing arithmetic.

So the stack must store numbers like:

```
2, 3, 4, 12, 14, 8, 4, 10
```

That means the stack must store `int` values.

So the main Lesson 20 change is:

```
char data;    // old version for characters
```

becomes:

```
int data;     // new version for numbers
```

3. Quick Review: What Is a Stack?

A stack is a linear data structure.

It follows this rule:

LIFO = Last In, First Out

This means:

The last value inserted is the first value removed.

Example:

```
push(10);  
push(20);  
push(30);
```

Logical stack view:

```
Top -> 30  
       20  
       10
```

The last value inserted was `30`.

So if we call:

```
pop();
```

The value removed is:

```
30
```

That is why a stack is useful in postfix evaluation.

When an operator appears, we need the most recent operands first.

4. Quick Review: Linked-List Stack

A linked-list stack stores data using connected nodes.

Example:

```
top -> 30 -> 20 -> 10 -> NULL
```

Here:

- `top` points to the first node.
- The first node contains `30`.
- So `30` is the top element.
- `20` is below `30`.
- `10` is at the bottom.
- The last node points to `NULL`.

In a linked-list stack:

```
top == NULL
```

means:

```
The stack is empty.
```

Example of an empty linked-list stack:

```
top -> NULL
```

5. Why We Use the Front of the Linked List as the Top

In a linked-list stack, the front node is used as the stack top.

Example:

```
top -> 30 -> 20 -> 10 -> NULL
```

If we push `40`, we add it at the front:

```
top -> 40 -> 30 -> 20 -> 10 -> NULL
```

Why do we use the front?

Because `top` already points to the front node.

So insertion and deletion at the front are fast.

- Push at the front: $O(1)$
- Pop from the front: $O(1)$

If we used the end of the linked list, we would need to scan the list first.

That would take:

```
 $O(n)$ 
```

So the front is the best place for stack operations.

6. What Is Postfix Evaluation?

Postfix evaluation means:

Calculating the final answer of a postfix expression.

Example:

35+

This means:

3 + 5

Final answer:

8

Another example:

82/

This means:

8 / 2

Final answer:

4

7. Why Postfix Evaluation Uses a Stack

A postfix expression gives operands first and operators later.

Example:

35+

When scanning from left to right:

1. We see 3.
2. We do not know the operator yet, so we store 3.

3. We see `5`.
4. We still do not know the operator yet, so we store `5`.
5. We see `+`.
6. Now we can add the two most recent numbers.

A stack is perfect here because it gives us the most recent values first.

So the stack is used to temporarily store numbers until an operator appears.

8. Main Rule of Postfix Evaluation

When scanning a postfix expression from left to right:

Symbol type	Action
Operand	Push it onto the stack.
Operator	Pop two values, calculate, and push the result.

Simple pseudocode:

```
for each symbol in the postfix expression:
    if the symbol is an operand:
        push it onto the stack

    if the symbol is an operator:
        x2 = pop()
        x1 = pop()
        r = x1 operator x2
        push(r)

final answer = pop()
```

At the end, the stack should contain one value.

That value is the final answer.

9. What Changes from Previous Linked-List Stack Programs?

In earlier linked-list stack programs, the node may have stored characters.

Example:

```
struct Node {  
    char data;  
    struct Node *next;  
};
```

This was useful for:

- Parenthesis matching
- Bracket matching
- Infix-to-postfix conversion

But in postfix evaluation, the stack stores arithmetic values.

So the node becomes:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

The stack rule does not change.

Only the data type changes.

10. Character Stack vs Integer Stack

Use case	Stack stores	Data type
Parenthesis matching	(,), [,], {, }	char
Infix-to-postfix conversion	+, -, *, /	char
Postfix evaluation	2, 3, 12, 14, 10	int

Important idea:

For postfix evaluation, the stack must store numbers, not symbols.

11. Node Structure for Lesson 20

The node structure is:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Meaning of each field:

Field	Simple meaning
data	Stores the integer value.
next	Points to the next node below it.

Example node:

```
[ data = 12 | next ]
```

This node stores the integer value `12`.

12. The Global `top` Pointer

We declare:

```
struct Node *top = NULL;
```

This creates a global pointer named `top`.

At the beginning, the stack is empty:

```
top -> NULL
```

After pushing values, the stack may look like this:

```
top -> 14 -> 8 -> 2 -> NULL
```

Here:

- `top` points to `14`.
 - `14` is the top value.
 - `8` is below `14`.
 - `2` is at the bottom.
-

13. Empty Condition

A linked-list stack is empty when:

```
top == NULL
```

Why?

Because `top` is a pointer.

If `top` points to nothing, there is no first node.

Example:

```
top -> NULL
```

This means:

```
The stack has no values.
```

14. Push Operation

Meaning

`push(int x)` inserts an integer at the top of the linked-list stack.

Example before pushing `30`:

```
top -> 20 -> 10 -> NULL
```

Call:

```
push(30);
```

After pushing:

```
top -> 30 -> 20 -> 10 -> NULL
```

The new value `30` becomes the top.

Push Code

```
void push(int x) {  
    struct Node *t;  
    t = (struct Node *)malloc(sizeof(struct Node));  
  
    if (t == NULL) {  
        printf("Stack Overflow\n");  
    } else {  
        t->data = x;  
        t->next = top;  
        top = t;  
    }  
}
```

Push Code Explanation

Step 1: Create a temporary pointer

```
struct Node *t;
```

`t` will point to the new node.

Step 2: Allocate memory for a new node

```
t = (struct Node *)malloc(sizeof(struct Node));
```

This asks the heap for memory for one new node.

Step 3: Check for overflow

```
if (t == NULL)
```

If `malloc()` returns `NULL`, memory was not available.

So we print:

```
Stack Overflow
```

Step 4: Store the value

```
t->data = x;
```

If we call:

```
push(12);
```

Then:

```
t->data = 12;
```

Step 5: Connect the new node to the old stack

```
t->next = top;
```

The new node points to the old top node.

Step 6: Move top to the new node

```
top = t;
```

Now the new node becomes the top of the stack.

15. The Key Push Pattern

The most important push pattern is:

```
t->next = top;  
top = t;
```

Meaning:

1. The new node points to the old top.
2. `top` moves to the new node.

The order matters.

Correct order:

```
t->next = top;  
top = t;
```

Wrong order:

```
top = t;  
t->next = top;
```

The wrong order can damage the stack.

It may make the new node point to itself or lose the old stack.

16. Pop Operation

Meaning

`pop()` removes and returns the top integer value.

Example before pop:

```
top -> 30 -> 20 -> 10 -> NULL
```

Call:

```
pop();
```

The removed value is:

30

After pop:

top -> 20 -> 10 -> NULL

Pop Code

```
int pop() {
    struct Node *p;
    int x = -1;

    if (top == NULL) {
        printf("Stack Underflow\n");
    } else {
        p = top;
        x = p->data;
        top = top->next;
        free(p);
    }

    return x;
}
```

Pop Code Explanation

Step 1: Create a temporary pointer

```
struct Node *p;
```

`p` will point to the node that will be removed.

Step 2: Create a return variable

```
int x = -1;
```

`x` stores the value to return.

If the stack is empty, `-1` is returned as a simple error value.

Step 3: Check underflow

```
if (top == NULL)
```

If `top == NULL`, the stack is empty.

There is no node to remove.

So we print:

```
Stack Underflow
```

Step 4: Save the old top node

```
p = top;
```

Now `p` points to the node that will be deleted.

Step 5: Save the data

```
x = p->data;
```

This saves the top value before deleting the node.

Step 6: Move top to the next node

```
top = top->next;
```

Now the second node becomes the new top.

Step 7: Free the old node

```
free(p);
```

This releases the memory used by the old top node.

Step 8: Return the saved value

```
return x;
```

The removed value is returned.

17. The Key Pop Pattern

The most important pop pattern is:

```
p = top;  
x = p->data;  
top = top->next;  
free(p);
```

Meaning:

1. Save the current top node in `p`.
2. Save its data in `x`.
3. Move `top` to the next node.
4. Free the old top node.
5. Return the saved value.

Important:

Do not free the node before saving the data.

Wrong idea:

```
free(p);  
x = p->data;    // wrong because p was already freed
```

Correct idea:

```
x = p->data;  
free(p);
```

18. What Is isOperand()?

The `isOperand()` function checks whether a symbol is an operand or an operator.

In this beginner lesson:

- Digits are operands.
- `+`, `-`, `*`, and `/` are operators.

The function used is:

```
int isOperand(char x) {  
    if (x == '+' || x == '-' || x == '*' || x == '/') {  
        return 0;  
    } else {  
        return 1;  
    }  
}
```

Meaning:

Return value	Meaning
1	The symbol is an operand.
0	The symbol is an operator.

Example:

```
isOperand('2')
```

returns:

```
1
```

because `2` is an operand.

Example:

```
isOperand('+')
```

returns:

0

because `+` is an operator.

19. Why the Program Uses `char *postfix`

The postfix expression is stored as a character string.

Example:

```
char postfix[] = "234*+82/-";
```

Inside this string, each symbol is a character:

```
'2' '3' '4' '*' '+' '8' '2' '/' '-' '\0'
```

The final character is:

```
'\0'
```

This is called the null terminator.

It marks the end of the string.

So the loop checks:

```
postfix[i] != '\0'
```

This means:

Keep scanning until the end of the string.

20. Character Digit vs Integer Value

This is a very important beginner concept.

In C, the character `'2'` is not the same as the integer `2`.

`'2'` is a character.

`2` is a number.

If we write:

```
push(postfix[i]);
```

and `postfix[i]` is `'2'`, we are not pushing the integer `2`.

We are pushing the character code for `'2'`.

That is not what we want.

For arithmetic, we need the real integer value.

So we write:

```
push(postfix[i] - '0');
```

21. How `postfix[i] - '0'` Works

Digit characters are stored in order:

```
'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'
```

So subtracting `'0'` gives the real number.

Examples:

```
'0' - '0' = 0  
'1' - '0' = 1  
'2' - '0' = 2  
'3' - '0' = 3  
'8' - '0' = 8
```

So when the program sees:

```
postfix[i] = '4'
```

this line:

```
postfix[i] = '0'
```

becomes:

```
4
```

That integer value is pushed into the stack.

22. Very Important: Pop Order Matters

When an operator appears, we pop two values.

The correct order is:

```
x2 = pop();    // right operand  
x1 = pop();    // left operand
```

Then we calculate:

```
r = x1 operator x2;
```

Why?

Because the first popped value comes from the top of the stack.

That value is the right operand.

The second popped value is the left operand.

Example: 82/

Postfix expression:

82/

This means:

8 / 2

Dry run:

Push 8
Push 2
See /

Now pop:

```
x2 = pop();    // x2 = 2  
x1 = pop();    // x1 = 8
```

Then:

```
r = x1 / x2;
```

So:

```
r = 8 / 2 = 4
```

Correct answer:

4

What Happens If We Reverse the Order?

Wrong code:

```
x1 = pop();  
x2 = pop();  
r = x1 / x2;
```

For `82/`, this would do:

```
2 / 8
```

That is wrong.

So always remember:

```
x2 = pop(); // first pop  
x1 = pop(); // second pop
```

23. Operators Affected by Pop Order

Addition and multiplication are less sensitive to order:

```
3 + 5 = 5 + 3  
3 * 5 = 5 * 3
```

But subtraction and division are very sensitive:

```
3 - 5 is not the same as 5 - 3  
8 / 2 is not the same as 2 / 8
```

So we must always use the correct pop order.

Even if addition and multiplication still work in reversed order, do not change the rule.

Always use:

```
x2 = pop();  
x1 = pop();  
r = x1 operator x2;
```

24. The Eval() Function

The Eval() function evaluates the postfix expression.

It receives a postfix string and returns the final integer answer.

Function header:

```
int Eval(char *postfix)
```

Meaning:

- postfix is the expression to evaluate.
- The function returns an integer answer.

Example:

```
Eval("234*+82/-")
```

returns:

```
10
```

25. Variables Used in Eval()

Inside Eval(), we use:

```
int i;  
int x1, x2, r;
```

Meaning:

Variable	Meaning
i	Scans the postfix string from left to right.
x2	Stores the first popped value, the right operand.
x1	Stores the second popped value, the left operand.

Variable	Meaning
<code>r</code>	Stores the result of the arithmetic operation.

Example:

For `82/`:

```
x2 = 2;
x1 = 8;
r = 8 / 2;
```

So:

```
r = 4;
```

26. `Eval()` Code

```
int Eval(char *postfix) {
    int i;
    int x1, x2, r;

    for (i = 0; postfix[i] != '\0'; i++) {
        if (isOperand(postfix[i])) {
            push(postfix[i] - '0');
        } else {
            x2 = pop();
            x1 = pop();

            switch (postfix[i]) {
                case '+':
                    r = x1 + x2;
                    break;

                case '-':
                    r = x1 - x2;
                    break;

                case '*':
                    r = x1 * x2;
                    break;
```

```

        case '/':
            r = x1 / x2;
            break;
    }

    push(r);
}

return pop();
}

```

27. Eval() Code Explanation

Step 1: Start scanning the postfix expression

```
for (i = 0; postfix[i] != '\0'; i++)
```

This scans every character until the null terminator.

Step 2: If the symbol is an operand

```
if (isOperand(postfix[i]))
```

If the symbol is a digit like '2', '3', or '8', it is an operand.

So we push it into the stack.

But first, we convert it from character to integer:

```
push(postfix[i] - '0');
```

Step 3: If the symbol is an operator

If the symbol is not an operand, it is an operator.

Example:

```
+  
-  
*  
/
```

Then we pop two values:

```
x2 = pop();  
x1 = pop();
```

Step 4: Apply the operator using `switch`

```
switch (postfix[i])
```

This checks which operator was found.

If it is `+`:

```
r = x1 + x2;
```

If it is `-`:

```
r = x1 - x2;
```

If it is `*`:

```
r = x1 * x2;
```

If it is `/`:

```
r = x1 / x2;
```

Step 5: Push the result back

After calculating, we push the result back into the stack:

```
push(r);
```

This is important because the result may be used by a later operator.

Step 6: Return the final answer

At the end, one value should remain in the stack.

So we return:

```
return pop();
```

That value is the final answer.

28. Why We Push the Result Back

Suppose we evaluate:

```
234*+
```

This means:

```
2 + (3 * 4)
```

First:

```
3 * 4 = 12
```

But we are not finished.

Now we still need:

```
2 + 12
```

So after calculating `12`, we must push it back into the stack.

If we do not push 12, the + operator will not have the correct value later.

That is why every operator step ends with:

```
push(r);
```

29. Full Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int x) {
    struct Node *t;
    t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        printf("Stack Overflow\n");
    } else {
        t->data = x;
        t->next = top;
        top = t;
    }
}

int pop() {
    struct Node *p;
    int x = -1;

    if (top == NULL) {
        printf("Stack Underflow\n");
    } else {
        p = top;
        x = p->data;
        top = top->next;
        free(p);
    }
}
```

```

        return x;
    }

    int isOperand(char x) {
        if (x == '+' || x == '-' || x == '*' || x == '/') {
            return 0;
        } else {
            return 1;
        }
    }
}

int Eval(char *postfix) {
    int i;
    int x1, x2, r;

    for (i = 0; postfix[i] != '\0'; i++) {
        if (isOperand(postfix[i])) {
            push(postfix[i] - '0');
        } else {
            x2 = pop();
            x1 = pop();

            switch (postfix[i]) {
                case '+':
                    r = x1 + x2;
                    break;

                case '-':
                    r = x1 - x2;
                    break;

                case '*':
                    r = x1 * x2;
                    break;

                case '/':
                    r = x1 / x2;
                    break;
            }

            push(r);
        }
    }

    return pop();
}

```

```
int main() {
    char postfix[] = "234*+82/-";

    printf("Result is %d\n", Eval(postfix));

    return 0;
}
```

Expected output:

Result is 10

30. Dry Run of 234*+82/-

Postfix expression:

234*+82/-

This expression means:

$2 + (3 * 4) - (8 / 2)$

Let us evaluate it step by step.

Initial stack:

empty

Step-by-Step Trace

Step	Symbol	Action	Stack after action
1	2	Operand, push 2	2
2	3	Operand, push 3	2, 3
3	4	Operand, push 4	2, 3, 4

Step	Symbol	Action	Stack after action
4	*	Pop 4, pop 3, calculate $3 * 4 = 12$, push 12	2, 12
5	+	Pop 12, pop 2, calculate $2 + 12 = 14$, push 14	14
6	8	Operand, push 8	14, 8
7	2	Operand, push 2	14, 8, 2
8	/	Pop 2, pop 8, calculate $8 / 2 = 4$, push 4	14, 4
9	-	Pop 4, pop 14, calculate $14 - 4 = 10$, push 10	10

Final stack:

10

Final answer:

10

31. Detailed Dry Run Explanation

Read 2

2 is an operand.

Convert it:

'2' - '0' = 2

Push it:

Stack: 2

Read 3

3 is an operand.

Convert it:

```
'3' - '0' = 3
```

Push it:

```
Stack: 2, 3
```

Read 4

4 is an operand.

Convert it:

```
'4' - '0' = 4
```

Push it:

```
Stack: 2, 3, 4
```

Read *

* is an operator.

Pop two values:

```
x2 = pop(); // 4  
x1 = pop(); // 3
```

Calculate:

```
r = x1 * x2;
```

So:

```
r = 3 * 4 = 12
```

Push the result:

```
Stack: 2, 12
```

Read **+**

+ is an operator.

Pop two values:

```
x2 = pop(); // 12  
x1 = pop(); // 2
```

Calculate:

```
2 + 12 = 14
```

Push the result:

```
Stack: 14
```

Read **8**

8 is an operand.

Convert it:

```
'8' - '0' = 8
```

Push it:

```
Stack: 14, 8
```

Read 2

2 is an operand.

Convert it:

```
'2' - '0' = 2
```

Push it:

```
Stack: 14, 8, 2
```

Read /

/ is an operator.

Pop two values:

```
x2 = pop(); // 2  
x1 = pop(); // 8
```

Calculate:

```
8 / 2 = 4
```

Push the result:

```
Stack: 14, 4
```

Read -

- is an operator.

Pop two values:

```
x2 = pop(); // 4
x1 = pop(); // 14
```

Calculate:

14 - 4 = 10

Push the result:

Stack: 10

Final answer:

10

32. Why the Stack Stores Intermediate Results

During evaluation, the stack does not only store original digits.

It also stores calculated results.

In the expression:

234*+82/-

The stack stores:

2
3
4
12
14
8
2
4
10

Notice that 12, 14, and 10 are not single characters from the original expression.

They are calculated integer results.

This is why the node must store:

```
int data;
```

not:

```
char data;
```

33. Why This Lesson Is About Reuse

Lesson 20 is not teaching a completely new data structure.

It is teaching reuse.

The linked-list stack still works the same way:

```
push = insert at top  
pop = remove from top  
top = first node  
empty = top == NULL
```

Only the stored data changes.

Old character stack:

```
struct Node {  
    char data;  
    struct Node *next;  
};
```

New integer stack:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

This is a powerful programming idea:

If the logic is reusable, only change the part that must change.

34. Time Complexity

Let n be the number of symbols in the postfix expression.

The program scans the postfix expression once.

For each symbol:

- If it is an operand, it does one push.
- If it is an operator, it does two pops and one push.

Each push is $O(1)$.

Each pop is $O(1)$.

So total time complexity is:

$O(n)$

Why?

Because every character is processed once.

35. Space Complexity

The stack may store many operands before an operator appears.

Example:

2345+++

The stack may grow as operands are pushed.

In the worst case, the stack can store up to n values.

So the space complexity is:

$O(n)$

The auxiliary variables such as `x1`, `x2`, `r`, and `i` use only constant space.

But the stack itself may grow with the expression size.

36. Complexity Summary

Part	Complexity	Reason
Scanning the postfix expression	$O(n)$	Each symbol is read once.
<code>push()</code>	$O(1)$	Insert at the front of linked list.
<code>pop()</code>	$O(1)$	Remove from the front of linked list.
Overall time	$O(n)$	One scan, constant-time stack operations.
Stack space	$O(n)$	Stack may store many values.

37. Limitations of This Beginner Program

This beginner version is useful, but it has some limitations.

Limitation 1: It handles single-digit operands only

It works for:

23+
82/
234*+

But it does not correctly handle multi-digit numbers like:

12 34 +

because it reads one character at a time.

So `12` would be read as:

'1' and '2'

not as:

12

Limitation 2: It assumes valid postfix input

The program assumes the postfix expression is correct.

For example:

23+

is valid.

But this is invalid:

2+

Why?

Because **+** needs two operands, but only one operand is available.

The beginner program may print underflow or behave incorrectly for invalid input.

Limitation 3: It does integer division

In C, if both operands are integers, division gives an integer result.

Example:

5 / 2

Result:

2

not:

2.5

So this program performs integer arithmetic.

Limitation 4: It does not handle spaces

This expression has no spaces:

234*+82/-

But expressions like this need extra handling:

2 3 4 * + 8 2 / -

The beginner version does not include that extra handling.

38. Common Beginner Mistakes

Mistake 1: Keeping `char data` in the node

Wrong:

```
struct Node {  
    char data;  
    struct Node *next;  
};
```

For postfix evaluation, this is not good because the stack must store results like `12` and `14`.

Correct:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Mistake 2: Pushing the character directly

Wrong:

```
push(postfix[i]);
```

If `postfix[i]` is `'2'`, this pushes the character code, not the integer `2`.

Correct:

```
push(postfix[i] - '0');
```

Mistake 3: Reversing pop order

Wrong:

```
x1 = pop();  
x2 = pop();  
r = x1 - x2;
```

Correct:

```
x2 = pop();  
x1 = pop();  
r = x1 - x2;
```

This is especially important for:

```
-  
/
```

Mistake 4: Forgetting to push the result

Wrong:

```
r = x1 + x2;
```

Correct:

```
r = x1 + x2;  
push(r);
```

The result must be pushed back because it may be used by the next operator.

Mistake 5: Forgetting to free memory in `pop()`

Wrong:

```
top = top->next;  
return x;
```

Correct:

```
p = top;  
x = p->data;  
top = top->next;  
free(p);  
return x;
```

If we do not call `free(p)`, removed nodes remain in memory.

That causes a memory leak.

Mistake 6: Thinking postfix needs precedence during evaluation

In postfix evaluation, we do not need precedence rules.

Why?

Because postfix already places operators in the correct execution order.

Example:

234*+

This already tells us:

3 * 4 first
then add 2

So postfix evaluation simply scans from left to right.

39. Mini Examples

Example 1: 23+

Expression:

23+

Dry run:

Symbol	Action	Stack
2	Push 2	2
3	Push 3	2, 3
+	Pop 3 and 2, calculate $2 + 3 = 5$, push 5	5

Answer:

5

Example 2: 82-

Expression:

82-

This means:

8 - 2

Dry run:

Symbol	Action	Stack
8	Push 8	8
2	Push 2	8, 2
-	Pop 2 and 8, calculate $8 - 2 = 6$, push 6	6

Answer:

6

Example 3: 93/

Expression:

93/

This means:

9 / 3

Dry run:

Symbol	Action	Stack
9	Push 9	9
3	Push 3	9, 3
/	Pop 3 and 9, calculate $9 / 3 = 3$, push 3	3

Answer:

3

Example 4: 23*54*+

Expression:

23*54*+

This means:

$(2 * 3) + (5 * 4)$

Dry run:

Symbol	Action	Stack
2	Push 2	2
3	Push 3	2, 3
*	Pop 3 and 2, calculate $2 * 3 = 6$, push 6	6
5	Push 5	6, 5
4	Push 4	6, 5, 4
*	Pop 4 and 5, calculate $5 * 4 = 20$, push 20	6, 20
+	Pop 20 and 6, calculate $6 + 20 = 26$, push 26	26

Answer:

26

40. Full Lesson Summary

Lesson 20 teaches how to reuse a linked-list stack for postfix evaluation.

The main points are:

1. A linked-list stack uses nodes and a `top` pointer.
2. In a linked-list stack, `top == NULL` means empty.
3. For postfix evaluation, the stack stores integers.
4. So the node must use `int data`.
5. When an operand is found, convert it using `postfix[i] - '0'` and push it.
6. When an operator is found, pop two values.
7. The first popped value is `x2`, the right operand.
8. The second popped value is `x1`, the left operand.
9. Calculate `x1 operator x2`.
10. Push the result back into the stack.
11. At the end, pop the final answer from the stack.
12. The expression `234*+82/-` evaluates to `10`.
13. Time complexity is `O(n)`.
14. Space complexity is `O(n)`.

41. One-Sentence Checkpoint

You understand Lesson 20 if you can explain this:

To evaluate a postfix expression using a linked-list stack, store integers in the nodes, push operands as integer values, pop `x2` first and `x1` second when an operator appears, calculate `x1 operator x2`, push the result back, and finally pop the answer.

42. Practice Questions

Practice 1

Evaluate:

23+

Expected answer:

5

Practice 2

Evaluate:

82 -

Expected answer:

6

Practice 3

Evaluate:

93 /

Expected answer:

3

Practice 4

Evaluate:

23 * 54 * +

Expected answer:

26

Practice 5

Evaluate:

234*+82/-

Expected answer:

10

43. Final Revision Checklist

Before moving on, make sure you can answer these questions:

- What does LIFO mean?
- What does `top` represent in a linked-list stack?
- What does `top == NULL` mean?
- Why does Lesson 20 use `int data` instead of `char data`?
- What does `push(int x)` do?
- What does `pop()` do?
- Why do we write `postfix[i] - '0'`?
- What does `isOperand()` return for a digit?
- What does `isOperand()` return for an operator?
- Why do we pop `x2` before `x1`?
- Why are subtraction and division sensitive to pop order?
- Why do we push the result back into the stack?
- What is the final answer of `234*+82/-`?
- What is the time complexity?
- What is the space complexity?

44. Final Takeaway

Lesson 20 is about combining ideas.

You already knew linked-list stacks.

You already knew postfix evaluation.

Now you combine them:

Linked-list stack + integer data + postfix evaluation rule = postfix evaluation using linked-list stack

The stack implementation is reused.

Only the stored data type changes.

That is the key idea of Lesson 20.